

Úvod: C a C++

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2023

Objektově-orientované programování, OOP 03/2023, Lekce 1

<https://moodle.vspj.cz/course/view.php?id=202513>



Vysoká škola
polytechnická
Jihlava

C versus C++

C – Brian Kernighan, Dennis Ritchie 1972 AT&T Bell Laboratories

- 1978 Kernighan, Ritchie: The C Programming Language

C++ – Bjarne Stroustrup 1983 AT&T Bell Laboratories

- 1985 Bjarne Stroustrup: The C++ Programming Language
- 1990 Bjarne Stroustrup: The Annotated C++ Reference Manual

C++ = C - nedovolené konstrukce

+ navíc neobjektová rozšíření

+ navíc objektová rozšíření

Doporučená literatura

1. Virius, Miroslav: **Programování v C++**. Od základů k profesionálnímu využití. Grada 2018, 518 str., ISBN 978-80-271-0502-1.
Přehledná a praktická základní učebnice C++.
2. Virius, Miroslav: **Jazyky C a C++**. Kompletní kapesní průvodce programátora. Grada 2011, 416 str., ISBN-13: 978-80-247-3917-5.
Jedná se spíše o referenční příručku, nežli o učebnici.
3. Lippman, Stanley, B. – Lajoie, Josée – Moo, Barbara, E.: **C++ Primer**, 5. vyd. Addison-Wesley 2015, ISBN-13: 978-0-321-71411-4.
Pozor: Nezaměňovat s C++ Primer Plus od S. Prata (to rozhodně ne)
4. Stroustrup, Bjarne: **Programming, Principles and Practice Using C++**, 2. vyd. Addison-Wesley 2014, ISBN-13 978-0-321-99278-9.
Velká kniha, bere to opravdu od začátku
5. Stroustrup, Bjarne: **Tour of C++**. Addison-Wesley 2013. ISBN-13 978-0-321-95831-0.
Tenký svazek, přehled toho co by měl C++ programátor umět

Přehled neobjektových rysů C++

- C++ je rozšíření proti C, ale C není podmnožinou C++
- změny ve funkcích:
 - zrušena možnost funkcí s libovolným počtem parametrů
 - nelze použít nedefinovanou funkci
 - `int f()` znamená `int f(void)`
 - funkce lze přetěžovat (jméno funkce nemusí být jednoznačné)
 - implicitní hodnoty parametrů
 - inline funkce
 - navíc parametry nahrazované odkazem (referencí)

Přetěžování (overloading) funkcí

- Funkce se rozlišují nejen jménem, ale i typy parametrů a počtem parametrů (profilem, aritou)

```
void f(char x) { printf("char %c\n", x); }
```

```
void f(int x) { printf("int %d\n", x); }
```

```
int main(void)
{
    f('a');
    f(10);
}
```

Přetěžování funkcí (pokračování)

- Při volání přetížené funkce se vyvolá ta, jejíž parametry se nejlépe “spárují” se skutečnými parametry
- Párování (matching) parametrů:
 1. Přesná shoda typů
 2. shoda typů po roztažení (promotion)
`char` → `int`, `short` → `int`, `enum` → `int`, `float` → `double`
 3. shoda typů po standardní konverzi
`int` → `float`, `float` → `int`, `int` → `unsigned`, ..., `int` → `long`
 4. shoda typů po uživatelské konverzi
- Pokud nejlepší párování má více než jedna funkce, ohlásí se chyba

Příklady přetěžování funkcí

```
void f(float, int);  
void f(int, float);  
f(1,1);           // chyba  
f(1.2,1.3);       // chyba
```

```
void f(long);  
void f(float);  
f(1.1);           // chyba  
f(1.1F);          // f(float)
```

```
void f(unsigned);  
void f(int);  
void f(char);  
unsigned char uc;  
f(uc);            // f(int)
```

Implicitní hodnoty parametrů

- V hlavičce funkce lze definovat implicitní (default) hodnotu parametru, skutečný parametr pak lze vynechat
- Skutečné parametry lze vynechávat od konce

```
void f(int x, int y=1, int z=3)
{ printf("x=%d, y=%d, z=%d\n", x, y, z); }
main() {
    f(10, 20, 30);
    f(10, 20);
    f(10);
}
```

- Co není dovoleno

```
f(int x=1, int y);           // chyba
f(int x, int y=10);
f(int x);
f(20);                       // nejednoznačnost
```


Inline funkce

- Volání **"inline"** funkce nemusí být skok do podprogramu, ale může být nahrazeno tělem funkce (posoudí překladač)

```
inline int max(int a, int b)
{ return (a>b)?a:b; }
```

```
int main()
{
    int a,b=3,c=4;
    . . .
    a=max(b,c) ;
}
```

Typ reference

- Proměnná typu reference je synonymum jiné proměnné

```
int i = 0;  
int& ir = i;           // ir ~ i  
ir = 2;                // i = 2
```

- Proměnná typu reference musí být inicializována

```
int &ir;                chyba
```

- Proměnná typu reference může být inicializována konstantou nebo proměnnou jiného typu (vytvoří se odkaz na pomocnou proměnnou)

```
int& ir = 6;  
float f = 3.14;  
int& ir2 = f;  
ir2 = 10;               // f se nezmění
```

Parametr typu reference

- Podobný efekt, jako parametr nahrazovaný odkazem

```
void swap1(int *x, int *y)
{ int z; z = *x; *x = *y; *y = z; }
```

```
void swap2(int& x, int& y)
{ int z; z = x; x = y; y = z; }
```

```
int main()
{
    int a = 10, b = 20;
    ...
    swap1(&a, &b);
    swap2(a, b);
    ...
}
```

Parametr typu reference (pokračování)

- Pro parametr typu $T\&$ je přípustným skutečným parametrem, kromě proměnné typu T , též:
 - konstanta typu T
 - proměnná nebo konstanta typu $T1$, který je kompatibilní vzhledem k přiřazení s T
- V těchto případech se vytvoří pomocná dočasná proměnná, do které se zkopíruje (s případnou konverzí) hodnota skutečného parametru, skutečný parametr se nezmění

```
float x=10.2;  
const int b=10;  
void g(int& x);  
g(x);  
g(b);
```

```
void f(int *x);  
f(&x); // chyba!  
f(&b); // chyba!
```

Konstantní parametr typu reference

- Pokud funkce nemění hodnotu skutečného parametru typu reference, je vhodné specifikovat parametr jako konstantní

```
struct Complex { float re, im; };  
Complex plus(const Complex& x, const Complex& y)  
{ Complex z;  
  z.re = x.re + y.re;  
  z.im = x.im + y.im;  
  return z;  
}
```

- Je-li skutečným parametrem konstanta, nevytvoří se dočasná proměnná

```
Complex a, b, const i = {0,1};  
a = plus(b, i);
```

Vstup a výstup

- Kromě standardních prostředků C lze vstup a výstup v C++ provádět pomocí datových proudů - objektů typu *stream*

```
cin          ~ stdin          cin  >> proměnná
cout         ~ stdout         cout << výraz
cerr        ~ stderr         cerr << výraz
#include <iostream>
int main()
{  int i;
   std::cout << "zadej i:";
   std::cin >> i;
   std::cout << "i=";
   std::cout << i;
   std::cout << '\n';
}
```

- možno zkrátit na: `std::cout << "i=" << i << '\n';`

Přípustné typy ve vstupu/výstupu

Vstup:

```
cin >> char
        short
        int
        long
        char *
        float
        double
        long double
```

Výstup:

```
cout << char
        short
        int
        long
        char *
        float
        double
        long double
        void *
```

Přeskočí počáteční mezery!

Manipulátory

- Slouží pro řízení vstupní a výstupní konverze
- Jsou definovány v souboru `<iomanip>`

```
int n = 36;  
cout << n << ', ' << hex << n << ', ';  
cout << oct << n << endl;  
cout << dec << setw(6) << n << endl;
```

- Některé manipulátory:

dec	dekadická konverze
hex	šestnáctková konverze
oct	osmičková konverze
endl	konec řádku + "flush"
setw(int n)	šířka položky n znaků
setfill(int c)	plnící znak c
setprecision(int n)	<i>n desetinných míst</i>

Konstanty

- Konstanta jednoduchého typu může být použita jako statický výraz

```
const int Max = 100;
int pole[Max];

int main()
{
    Max = 10;           // chyba!
    int *p;
    p = &Max;           // chyba!
    const int *q;
    q = &Max;
}
```

Deklarace je příkaz

- Deklarace se může vyskytovat mezi příkazy
- Deklarace se může vyskytovat v příkazu **for**

```
int main()
{
    for (int i=0; i<10; i++)
    { ... }
    for (int i=10; i>0; i--) // chyba! druhá deklarace
    { ... }
}
```

Skok za deklaraci

- Nelze přeskočit deklaraci s inicializací

<code>goto lab2;</code>	<code>goto lab2; // chyba!</code>
<code>...</code>	<code>...</code>
<code>lab1:</code>	<code>lab1:</code>
<code>int a;</code>	<code>int a = 10;</code>
<code>...</code>	<code>...</code>
<code>lab2:</code>	<code>lab2:</code>
<code>...</code>	<code>...</code>
<code>goto lab1;</code>	<code>goto lab1;</code>

Jména struktur

- Jména struktur jsou jmény typů a patří do stejného prostoru jmen, jako jména proměnných, funkcí atd. (totéž platí pro **union** a **enum**)

C++:

```
struct Prvek {  
    int    hodn;  
    Prvek *dalsi;  
};  
  
Prvek *zacatek;  
  
// V C++ není nutné
```

C:

```
struct Prvek {  
    int    hodn;  
    struct Prvek *dalsi;  
};  
  
struct Prvek *zacatek;  
    nebo  
typedef struct Prvek Prvek;  
/* a pak je možné */  
Prvek *zacatek;
```

Dynamické proměnné

- Pro vytvoření dynamické proměnné se nepoužívá funkce `malloc`, ale operátor `new`

```
int *p = new int;  
struct S {int a; char b;};  
S *q = new S;
```

- Operátorem `new` lze vytvořit pole

```
int Pocet = 10;  
int *pa = new int[Pocet];
```

- Pro zrušení dynamické proměnné slouží operátor `delete`

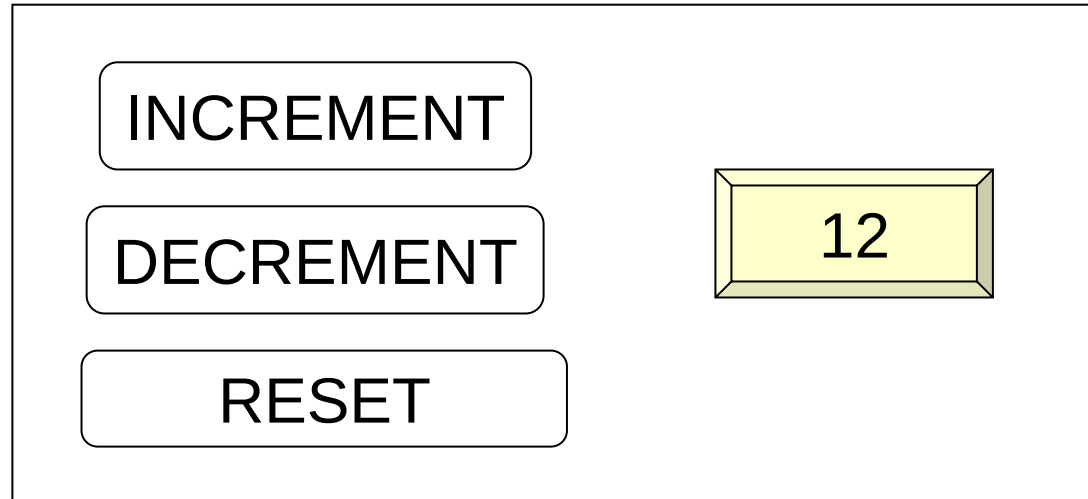
```
delete p;  
delete [] pa;
```

- Operátorem `delete` lze zrušit jen to, co bylo vytvořeno operátorem `new`

```
p = pa + 1; delete p;           // chyba
```

Programovací styly

Příklad: Čítač



Řešení v různých stylech

- naivní
- procedurální
- modulární
- objektově orientovaný

Návrh komunikace s programem

Hodnota=0

1. Increment
2. Decrement
3. Reset
4. Konec

Vase volba:

1

Zadává
uživatel

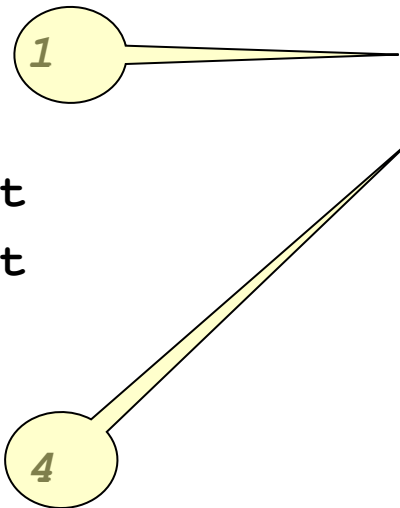
Hodnota=1

1. Increment
2. Decrement
3. Reset
4. Konec

Vase volba:

4

Konec



Naivní styl

**Z jakýchkoli
(strukturovaných)
konstrukcí sestav
řešení**

```
#include <iostream>
using namespace std;
const int PocHodn = 0;
int main() {
    int Hodn; char Volba;
    Hodn = PocHodn;
    do {
        cout << "Hodnota = " << Hodn << endl << endl;
        cout << "1. Increment\n";
        cout << "2. Decrement\n";
        cout << "3. Reset\n";
        cout << "4. Konec\n";
        cout << "Vase volba: ";
        cin >> Volba;
        cin.ignore(100, '\n');
        switch (Volba) {
            case '1': Hodn++; break;
            case '2': Hodn--; break;
            case '3': Hodn = PocHodn; break;
            case '4': break;
            default: cout << "Nedovolena volba\n";
        }
    } while (Volba != '4');
    cout << "Konec\n";
}
```


Procedurální styl

**Rozlož problém
na
podproblémy.
Navrhni řešení
podproblémů
jako procedury
(funkce). S
jejich pomocí
sestav řešení.**

```
const int PocHodn;  
int Hodn;  
int Menu()  
    /* vypíše nabídku a Přečte volbu  
       vrátí: 0=konec, 1,2,3=operace s čítačem  
    */  
    { ... }  
void Citac(int op)  
    /* provede operaci op s čítačem Hodn */  
    { ... }  
int main()  
{   int Volba;  
    Hodn = PocHodn;  
    do {  
        cout << "Hodnota=" << Hodn << endl;  
        Volba = Menu();  
        if (Volba>0) Citac(Volba);  
    } while (Volba>0);  
    cout << "Konec\n";  
}
```

Modulární styl

**Rozlož problém na podproblémy.
Navrhni vhodné datové typy.
Specifikuj operace nad nimi,
implementuj je pomocí procedur a funkcí. "Ukryj" definice typu do modulu.**

Zavedeme dva moduly: Citac a Menu

```
#include <iostream>
using namespace std;
#include "menu.h"
#include "citac.h"
int main()
{ InitMenu(); InitCitac(0);
  do {
    cout << ...;
    switch (GetUserSelection()) {
      case 1: Increment(); break;
      case 2: Decrement(); break;
      case 3: Reset(); break;
    }
  } while (GetSelection() != 0);
  DoneMenu(); cout << "Konec\n";
}
```

Realizace typu modulem

- Modul má dvě části:
 - specifikační část (specifikuje vyvážené prostředky)
 - implementační část (ukrývá implementaci)
- Specifikační část v C++: soubor s příponou **.h** (příp. **.hpp**)
- Implementační část v C++: soubor s příponou **.cpp**
- Návrh modulu závisí na tom, zda stačí jediná instance typu nebo zda má umožňovat práci s více instancemi (proměnnými) daného typu
- Typ v jediné instanci:
 - modul vyváží pouze operace, instance je ukryta v implementační části
- Typ pro více instancí:
 - modul vyváží **typ** (aby bylo možné deklarovat proměnné tohoto typu) a **operace**, které jsou parametrizovány typem (aby bylo možné určit proměnnou, s níž se operace provede)

Specifikace “Menu” pro jedinou instanci

```
/* menu.h */  
void InitMenu();  
void DoneMenu();  
void AddItem(char *txt);  
void AddPrompt(char *txt);  
int GetUserSelection();  
int GetSelection();
```

Implementace “Menu”

pro jedinou instanci - reprezentace dat

```
/* menu.cpp */  
#include <iostream>  
using namespace std;  
#include <string.h>  
#include "menu.h"
```

**Jenom naše
programátorská
disciplina!**

```
static const int MaxPrvku = 9;  
static char *Prvky[MaxPrvku];  
static int PocetPrvku;  
static char *Vyzva;  
static int Volba;
```

Implementace “Menu”

pro jedinou instanci - pomocná funkce **NewStr**

```
static char *NewStr(char *str)
{
    char *p;
    p = new char[strlen(str) + 1];
    strcpy(p, str);
    return p;
}
```

Lokální v modulu Menu

Implementace “Menu”

pro jedinou instanci - **InitMenu** a **DoneMenu**

```
void InitMenu() {
    PocetPrvku = 0;
    Vyzva = NULL;
}

void DoneMenu() {
    for (int i = 0; i < PocetPrvku; i++) {
        delete [] Prvky[i];
        Prvky[i] = NULL;
    }
    if (Vyzva) {
        delete [] Vyzva;
        Vyzva = NULL;
    }
}
```

Implementace “Menu”

pro jedinou instanci - **AddItem**, **AddPrompt** a **GetSelection**

```
void AddItem(char *txt)
{
    if (PocetPrvku < MaxPrvku)
        Prvky[PocetPrvku++] = NewStr(txt);
}

void AddPrompt(char *txt)
{
    Vyzva = NewStr(txt);
}

int GetSelection()
{
    return Volba;
}
```


Implementace “Menu”

pro jedinou instanci - **GetUserSelection**

```
int GetUserSelection()
{ char zn;
  cout << endl;
  do {
    for (int i=0; i<PocetPrvku; i++) cout << Prvky[i] <<
endl;
    cout << Vyzva;
    cin >> zn; cin.ignore(100, '\n');
    if (zn >= '1' && zn <= '9') {
      Volba = zn - '0';
      if (Volba == PocetPrvku) Volba = 0;
      else if (Volba > PocetPrvku) Volba = -1;
    } else Volba = -1;
    if (Volba < 0) cout << "Nedovolena volba\n";
  } while (Volba < 0);
  return Volba;
}
```

Program s jedinou instancí “Menu”

```
#include "menu.h"
#include "citac.h"
int main()
{ cout << "\n\nCITAC\n";
  InitMenu(); InitCitac(0);
  AddItem("1. Increment"); AddItem("2. Decrement");
  AddItem("3. Reset"); AddItem("4. Konec");
  AddPrompt("Vase volba: ");
  do {
    cout << "Hodnota = " << GetValue() << endl;
    switch (GetUserSelection()) {
      case 1: Increment(); break;
      case 2: Decrement(); break;
      case 3: Reset(); break;
    }
  } while (GetSelection() != 0);
  DoneMenu(); cout << "Konec\n";
}
```

Specifikace modulu “Čítač”

pro více instancí (vyvážející typ)

```
/* citac.h */  
struct Citac {  
    int Hodn;  
    int PocHodn;  
};  
void Increment(Citac &c);  
void Decrement(Citac &c);  
void Reset(Citac &c);  
int GetValue(Citac &c);  
void InitCitac(Citac &c, int ph);
```

Citac
int Hodn int pocHodn

Implementace modulu “Čítač”

pro více instancí

```
/* citac.cpp */
```

```
#include "citac.h"
```

```
void Increment(Citac &c)
```

```
{ c.Hodn++; }
```

```
void Decrement(Citac &c)
```

```
{ c.Hodn--; }
```

```
void Reset(Citac &c)
```

```
{ c.Hodn = c.PocHodn; }
```

```
int GetValue(Citac &c)
```

```
{ return c.Hodn; }
```

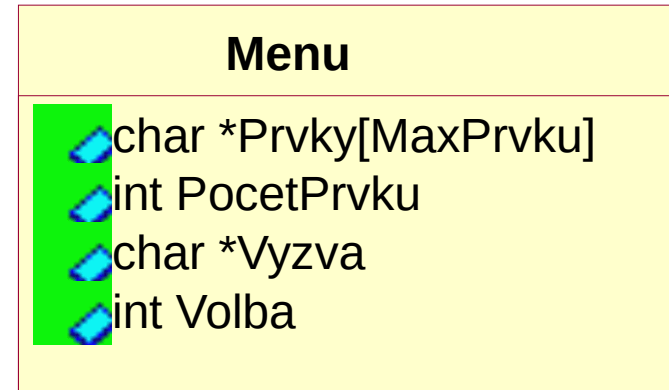
```
void InitCitac(Citac &c, int ph)
```

```
{ c.PocHodn = ph; Reset(c); }
```

**Jenom naše
programátorská
disciplina!**

Specifikace modulu “Menu”

```
/* menu.h */                                pro více instancí
/* Specifikace typu Menu pro více instancí*/
const int MaxPrvku = 9;
struct Menu {
    char *Prvky[MaxPrvku] ;
    int  PocetPrvku;
    char *Vyzva;
    int  Volba;
};
void InitMenu(Menu &m) ;
void DoneMenu(Menu &m) ;
void AddItem(Menu &m, char *txt) ;
void AddPrompt(Menu &m, char *txt) ;
int  GetUserSelection(Menu &m) ;
int  GetSelection(Menu &m) ;
```



Implementace modulu “Menu”

pro více instancí - **InitMenu**

```
/* menu.cpp */
/* Implementace typu Menu pro vice instanci */
#include <iostream>
using namespace std;
#include <string.h>
#include "menu.h"
void InitMenu(Menu &m)
{
    m.PocetPrvku = 0;
    m.Vyzva = NULL;
}
```

Implementace modulu “Menu”

pro více instancí - **DoneMenu**

```
void DoneMenu(Menu &m)
{
    for (int i = 0; i < m.PocetPrvku; i++) {
        delete m.Prvky[i];
        m.Prvky[i] = NULL;
    }
    if (m.Vyzva) {
        delete m.Vyzva;
        m.Vyzva = NULL;
    }
}
```

Program s proměnnými typu Čítač a Menu

```
/* main.cpp */
#include "menu.h"
#include "citac.h"
void main(void)
{   Citac citac; Menu menu;
    cout << "\n\nPRIKLAD 4\n";
    InitMenu(menu); InitCitac(citac, 0);
    AddItem(menu, "1. Increment");
    ...
    do {
        cout << "Hodnota = " << GetValue(citac) << endl;
        switch (GetUserSelection(menu)) {
            case 1: Increment(citac); break;
            case 2: Decrement(citac); break;
            case 3: Reset(citac); break;
        }
    } while (GetSelection(menu) != 0);
    DoneMenu(menu);
    cout << "Konec\n";
}
```


Objektově orientovaný styl

**Rozlož problém na podproblémy.
Navrhni potřebné třídy objektů a realizuj všechny operace tříd.**

```
/* menu.h */
/* Objektově orientovaná specifikace
   typu Menu */
const int MaxPrvku = 9;
class Menu {
    char *Prvky[MaxPrvku];
    char PocetPrvku;
    char *Vyzva;
    int Volba;
public:
    void AddItem(char *txt);
    void AddPrompt(char *txt);
    int GetUserSelection();
    int GetSelection();
    Menu();
    ~Menu();
};
```

Deklarace třídy (`class`)

- Typ třída = datové položky + metody + konstruktory + destruktory

```
class T {  
    deklarace položek  
    prototypy metod          typ jméno( ... );  
    prototypy konstruktorů   T( ... );  
    prototyp destruktoru     ~T();  
};
```

- Definice metody:

```
typ T::jméno( ... ) { ... }
```

- Definice konstruktoru:

```
T::T( ... ) { ... }
```

- V těle metody (konstruktoru, destruktoru) jsou přímo vidět položky (metody) daného objektu

Specifikace třídy (uživatelského typu) „Menu“

```
/* menu.h */
/* Specifikace tridy Menu */
const int MaxPrvku = 9;
class Menu {
    char *Prvky[MaxPrvku];
    int PocetPrvku;
    char *Vyzva;
    int Volba;
public:
    Menu(Menu &m);
    ~Menu(Menu &m);
    void AddItem(char *txt);
    void AddPrompt(char *txt);
    int GetUserSelection();
    int GetSelection();
};
```

Implementace třídy Menu

konstruktor

```
/* menu.cpp */  
#include <iostream>  
using namespace std;  
#include <string.h>  
#include "menu.h"  
Menu::Menu()  
{  
    PocetPrvku = 0;  
    Vyzva = NULL;  
}
```

Implementace třídy Menu

destruktor

```
Menu::~Menu()
{
    for (int i = 0; i < PocetPrvku; i++) {
        delete [] Prvky[i];
        Prvky[i] = NULL;
    }
    if (Vyzva) {
        delete [] Vyzva;
        Vyzva = NULL;
    }
}
```

Implementace třídy Menu (pokrač.)

```
void Menu::AddItem(char *txt)
{
    if (PocetPrvku < MaxPrvku)
        Prvky[PocetPrvku++] = NewStr(txt);
}
void Menu::AddPrompt(char *txt)
{
    Vyzva = NewStr(txt);
}
int Menu::GetUserSelection()
{
    ...
}
int Menu::GetSelection()
{
    return Volba;
}
```

Třída Čítač

```
/* citac.h */
/* Objektově orientována specifikace
   a implementace tridy Citac */
class Citac {
    int    Hodn;
    int    PocHodn;
public:
    Citac(int ph)    {PocHodn = ph; Reset();}
    ~Citac()         {} // lze vypustit, udělá se sám
    void Increment() {Hodn++;}
    void Decrement() {Hodn--;}
    void Reset()     {Hodn = PocHodn;}
    int GetValue()   {return Hodn;}
};
```

Program s objekty třídy Čítač a Menu

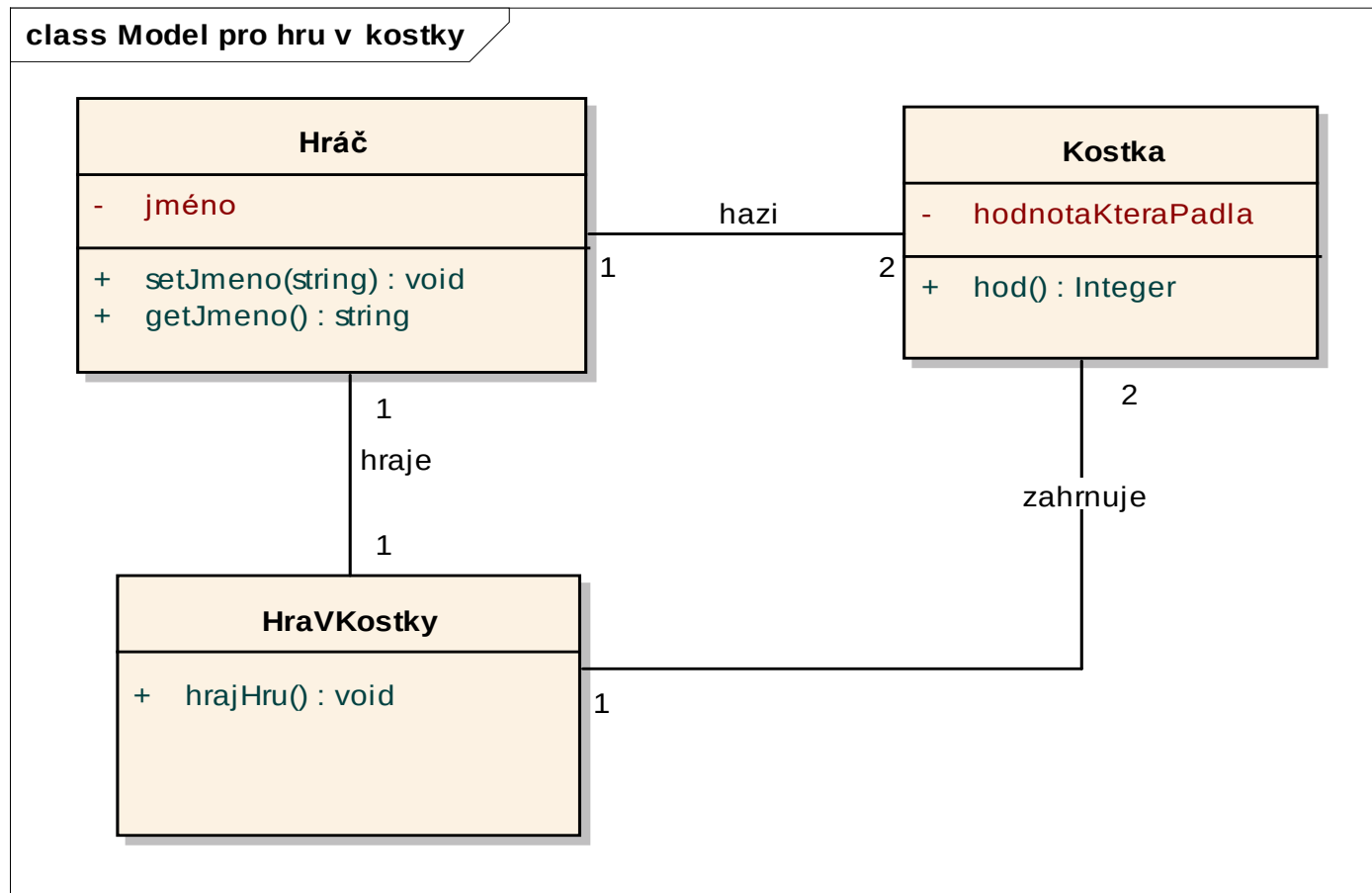
```
/* main1.cpp */
#include "menu.h"
#include "citac.h"
void main()
{ Citac citac(0); Menu menu;
  cout << "\n\nCITAC - Objektově orientovaný styl\n";
  menu.AddItem("1. Increment"); menu.AddItem("2. Decrement");
  menu.AddItem("3. Reset"); menu.AddItem("4. Konec");
  menu.AddPrompt("Vase volba: ");
  do { cout << "Hodnota = " << citac.GetValue() << endl;
      switch (menu.GetUserSelection()) {
        case 1: citac.Increment(); break;
        case 2: citac.Decrement(); break;
        case 3: citac.Reset(); break;
      }
    } while (menu.GetSelection() != 0);
  cout << "Konec\n";
}
```


Příklad: Hra v kostky

- Uvažme příklad velmi jednoduché hry s kostkami – máme dvě kostky a házíme. Pokud nám vyjde součet v daném hodu 7, vyhráli jsme, jinak jsme naopak prohráli.
- Objektové řešení vychází z podstaty úlohy – pracujeme s objekty typu „kostka“. To jsou instance obecné třídy „Kostka“.
- Kostka umí metodu „hod“, která nastaví interní hodnotu kostky „hodnotaKteraPadla“ na náhodné číslo a vrátí toto číslo jako výsledek.
- Hru hrají hráči, kteří mají jméno – to je možno jim pomocí metod nastavit a číst (setJmeno/getJmeno).
- Hra spočívá v tom, že hráč hodí kostkami – to nám může zajistit metoda „hrajHru“.



Objektový model pro hru v kostky



Třída „Kostka“

```
class Kostka {  
private:  
    int hodnotaKteraPadla;  
public:  
    int hod() { return hodnotaKteraPadla = „náhodná hodnota“; }  
};
```

```
#include <stdlib.h>  
class Kostka {  
private:  
    int hodnotaKteraPadla;  
public:  
    int hod() { return hodnotaKteraPadla = (rand() % 6) + 1; }  
};
```

Třída „Hráč“

```
#include <string>
```

```
class Hrac {
```

```
private:
```

```
    string jméno;
```

```
public:
```

```
    void setJmeno(string jm) { jméno = jm; }
```

```
    string getJmeno() { return jméno; }
```

```
};
```

Třída „HraVKostky“

```
#include <stdlib.h>
#include <time.h>

class HraVKostky {
private:
    Kostka kostka1;
    Kostka kostka2;
public:
    HraVKostky() { srand((unsigned)time(NULL)); }
    /* nastavení náhodného generátoru */
    int hrajHru() {
        return kostka1.hod() + kostka2.hod();
    }
};
```

Hlavní program „main“ pro kostky

```
int main() {
    HraVKostky Hra;
    Hráč h;
    string jmeno;
    int vysledek;
    char volba;
    cout << "Hra v kostky" << endl;
    cout << "Vase jmeno: "; cin >> jmeno; h.setJmeno(jmeno);
    cout << "Nazdar " << h.getJmeno() << endl;
    do {
        vysledek = Hra.hrajHru();
        if (vysledek == 7) cout << "Vyhrál jsi!!!" << endl;
        else cout << "Prohrál jsi, sorry, součet je: " << vysledek <<
endl;
        cout << "Pokracovat?: "; cin >> volba;
    } while (volba != 'n');
    return 0;
}
```



The End